How Far Are We from Detecting Flaky Tests? On the Limits of Code-Based Detection

Ömer Oktay Gültekin [1], Alexander Berndt [1,3], Jonathan Bell [2], Thomas Bach [3], and Sebastian Baltes [1]
[1]Heidelberg University, Germany [2]Northeastern University, USA [3]SAP, Germany
oemer.gueltekin@stud.uni-heidelberg.de, alexander.berndt@uni-heidelberg.de, j.bell@northeastern.edu,
thomas.bach03@sap.com, sebastian.baltes@uni-heidelberg.de

arXiv:2607.09345v1 [cs.SE] 10 Jul 2026

***Abstract*—Flaky tests pass and fail on the same code version, weakening the signal of test results and disrupting continuous integration (CI) pipelines. Code-based flakiness detectors report strong benchmark results, yet their use in practice remains limited. We argue that the field is studying the wrong problem: Flakiness is not a static property of test code, which often lacks the information needed to decide whether a test is flaky. Analyzing three code-based detectors operating on test code, we found that widely used benchmarks contain shortcuts that inflate reported F1 scores and that evaluation protocols overstate generalizability. To control for these shortcuts, we curated two datasets. The first, `C-IDoFT` (54 468 unit tests from 57 GitHub projects), keeps a developer-confirmed subset of `IDoFT`'s flaky tests and rebuilds only the non-flaky class from repeated executions instead of fixed versions of flaky tests. `C-IDoFT` is a controlled counterfactual, not a benchmark for reuse. Our CodeBERT reimplementations of two published detectors scored far above its constant baselines under the published cross-validation protocol but no better than them once projects were separated. The high scores rested on the labeling shortcut and the evaluation protocol, not on the test code. On `FlakeBench`, a benchmark restricted to flakiness types typically recognizable from test code, and the same project-disjoint protocol, the models identified nearly all flaky tests. The second dataset, mined from CI logs, contains 86 flaky end-to-end tests that passed and failed on the same commit. The test code and CI log yielded a cause for 42% of them; the other 58% required further execution evidence. Rather than abandoning flakiness prediction, we reframe it around whether an observed *failure* is flaky and how likely a *test* is to fail given its execution environment. Our datasets and CI-mining method support this direction.**

I. Introduction

Flaky tests impede continuous integration (CI). Flaky tests pass and fail on the same version of the code, so their failures no longer reliably signal a software defect [1]. Thus, flaky tests waste developer time spent debugging spurious failures and erode trust in test results [2]. The common mitigation is to repeatedly re-execute failing tests in the CI pipeline to separate flaky failures from real ones. Companies such as Google rely on this to keep flaky failures from blocking their pipelines [3], and Meta reports that flakiness reduces test effectiveness and increases the cost of testing [4]. However, repeatedly executing flaky tests is expensive at scale, and at Google flaky tests even undermine rerun-based test selection heuristics [5]. For example, practitioners at SAP report that 500 hours of compute per day go to rerun-based handling of flaky failures during SAP HANA pre-submit testing on the main branch alone [6].

To avoid the cost of repeatedly executing flaky tests, a large body of work tries to detect flaky tests statically, without reruns, using signals such as the test execution history [3, 7, 8], code metrics [9], or combinations of metrics and the test code [10]. The most actively studied approach is code-based detection, where a model is trained on a test's source code to predict whether it is flaky, ranging from Pinto et al.'s lexical approach and follow-up vocabulary studies [11–14] through feature-based and fine-tuned code models [15–17] to recent methods based on Large Language Models (LLMs) [18, 19], the strongest of which report F1 scores above 0.90 on standard benchmarks. Yet this success has not reached practice, and industrial adoption remains limited [19, 20]. For example, in a previous industrial study, a code-based detector was evaluated but never deployed because it did not generalize to real-world data [13]. Recent work has begun to explain why the benchmark numbers are fragile: Some are inflated by experimental flaws [17, 19], and the test code alone may not contain the information needed to separate flaky from non-flaky tests [19].

We argue that these fragilities are symptoms of studying the wrong problem, not isolated defects to be patched. Flakiness is not a static property of a test's source code: It arises from how the test interacts with the system under test and its execution environment at run time. Asking "is this test flaky?" from the code alone is therefore ill-posed for many kinds of flakiness, no matter how the detector is built. Rahman et al. traced one detector's inflated score to a data-leakage flaw in its cross-validation [17]. We additionally found that the same data leakage affected that detector's published leave-one-project-out evaluation. In this work, we broaden the critique from a single implementation defect to the task framing itself, and we reproduce published approaches under a single, rigorous protocol, project-disjoint wherever the approach permits it, showing that the failure of code-based detection persists across benchmarks and models.

We proceed in two steps. First, we reproduce three code-based detectors published at major software engineering venues [16, 18, 21] and audit their artifacts and benchmarks. We find that commonly used benchmarks contain shortcuts that inflate the reported F1 scores and that the evaluation protocols overstate generalizability. To control for these shortcuts, we build `C-IDoFT`, a dataset of 54 468 unit tests from 57 projects, with flaky labels from developer-confirmed tests in

the International Dataset of Flaky Tests (IDoFT) and non-flaky labels from repeated execution rather than from fixed versions of flaky tests (the *fix-commit* shortcut). Where an earlier critique demonstrated the problem on a newly constructed dataset [17], C−IDoFT keeps the developer-confirmed flaky tests those detectors were evaluated on and rebuilds only their non-flaky counterparts from reruns. After the shortcut was removed and projects were separated, the detectors did no better than a majority-class baseline, a constant predictor that always outputs the more frequent class. However, we argue that this weak performance is caused by the ill-posed framing of the task, and view C−IDoFT as a control rather than a benchmark we recommend adopting.

Second, we analyze whether CI logs, in addition to the test code, can be used as a signal to identify flaky failures. We mine more than 450 000 CI runs from open-source GitHub projects to build FlakeCI, a set of flaky end-to-end (E2E) tests that pass and fail on the same commit. Based on this dataset, we could attribute a cause to 42% of these tests from the test code and CI log together. For the remaining tests, diagnosis required execution evidence that neither source provided.

To summarize, these results relocate the problem: They move flaky test detection from the test's code to its execution and ask whether an observed *failure* is flaky or how likely a *test* is to fail. Once the benchmark shortcuts are removed and projects are separated, code-based detectors are, on developer-confirmed flakiness with rerun-confirmed non-flaky labels, no closer to detecting flaky tests than a majority-class baseline, and the progress reported in prior studies largely rested on those shortcuts and the cross-validation protocol. We argue that this failure-level formulation is where the field can make genuine progress.

The contributions of this work are as follows:

1) A critical analysis showing that the apparent success of code-based flaky test detection is largely an artifact of how the task and its benchmarks are framed, demonstrated by reproducing three published detectors.
2) C−IDoFT, a controlled counterfactual that rebuilds the non-flaky class of the IDoFT benchmark used in prior studies, removing the fix-commit shortcut.
3) FlakeCI, a CI-log-mined corpus of confirmed flaky E2E failures, with evidence that for most of its tests, a cause could not be attributed from the test code and the CI log alone.

The remainder of this paper is structured as follows: Section II introduces background on flaky test detection and the studies we reproduce. Section III introduces our research questions and methodology. Section IV presents our empirical results. We discuss the results in Section V and threats to their validity in Section VI. Section VII discusses related work. Section VIII concludes this work. Our replication package is available [22].

II. Background

A. Flaky Test Detection

Previous work, surveyed systematically by Parry et al. [23], proposed approaches that detect flaky tests statically, i.e., without repeated executions. These approaches can be grouped by the data they use for detection: the test execution history [3, 7], metrics of the test such as lines of code or execution time [9, 15], or the test's source code itself, from vocabulary-based models [11, 24] and their replications [12, 13, 25] to CodeBERT-based detectors [10, 18, 21]. In this work, we focus on the latter group, the code-based approaches operating on the test's source code, from the vocabulary-based models to the CodeBERT-based detectors, which have recently received increasing attention due to their promising results across various benchmark datasets.

The intuition behind code-based flaky test detection is that flaky tests exhibit syntactical patterns that distinguish them from non-flaky tests [11]. After Pinto et al. introduced the approach in 2020 [11], replications revealed generalizability issues. Performance degraded on newer revisions of a repository (up to a 21% drop in MCC) [12] and on unseen projects [25]. An industrial evaluation concluded that this lack of generalizability prevents production deployment [13]. Therefore, more recent work has proposed using LLMs for flaky test detection [16, 18, 21], with Flakify explicitly motivated by these generalizability issues. Fatima et al. extended this line to predicting flaky-test fix categories with LLMs [26].

B. Datasets

The datasets used to evaluate code-based flaky test detection share a common lineage but differ along two independent axes in how their flaky and non-flaky labels are obtained. On the flaky axis, tests are either (i) mined from community reports or curated tool runs, (ii) drawn from an expert-curated category set, (iii) detected by reruns, or (iv) mined from CI logs (the strategy behind our FlakeCI dataset, see Section III). On the non-flaky axis, a test is labeled non-flaky in one of two ways: (i) the test is a developer-fixed later version of a reported flaky test (*fix-commit*), or (ii) it survives many repeated executions (*rerun-confirmed*). Each dataset pairs one flaky source with one non-flaky source. We organize the datasets below by that pairing and describe them in chronological order.

msr4flakiness. Pinto et al. constructed the first code-based flakiness dataset by running every test case of 24 projects from the DeFlaker benchmark [27] 100 times, a total of 64 000 tests and roughly 6.4 million test executions. Tests that did not change their outcome across reruns were labeled non-flaky [11]. This dataset uses rerun-confirmed non-flaky labels with a 100-run budget. Its flaky labels are inherited from the DeFlaker dataset.

FlakeFlagger. FlakeFlagger selected 24 projects previously studied in flaky test research, 17 of which overlap with those of msr4flakiness [15]. To obtain rerun-based labels with a deeper budget, the authors re-executed each project's test suite 10 000 times. They found that even 10 000 reruns

did not detect all flaky tests, and that shallower budgets miss many tests that fail only rarely. As reported by Fatima et al., the resulting dataset contains 802 flaky and 20 859 non-flaky tests across 23 projects, totaling 21 661 tests [16]. In addition to rerun-based labels, `FlakeFlagger` provides features such as test smells, coverage, and execution time. Because both its flaky and non-flaky labels come from the same deep rerun campaign rather than from developer fixes or an externally sourced flaky list, `FlakeFlagger` is the most realistic of the public datasets and the one we do *not* modify in this work.

IDoFT. The International Dataset of Flaky Tests (`IDoFT`) is an open-source GitHub repository tracking flaky tests in open-source Java and Python projects [28, 29]. Its flaky tests stem from research detection tools in the lineage of `iDFlakies`, which detects flaky tests by rerunning a project's tests under reordered execution orders, and from community submissions [29]. `IDoFT` itself lists only flaky tests, together with each test's fix status. The classification dataset derived from it, introduced by `Flakify` [16], adds *fix-commit* non-flaky labels. A non-flaky test is the fixed version of a flaky test, obtained from the pull request linked in `IDoFT`, where the fix changed the test code itself. The variant used to evaluate `Flakify` and its successors consists of 3813 tests, of which 3195 are flaky and 618 are non-flaky [21].[1] As we discuss in Section III, this fix-commit construction yields two artifacts: a non-flaky class far smaller than the flaky class, and fixed/unfixed test pairs that are nearly identical in their code.

FlakeBench. `FlakeBench` [17] was built to give `FlakyCat`'s flaky tests a realistic non-flaky population. `FlakyCat` is a manually curated dataset of categorized flaky tests [30] whose categories derive from root causes defined in prior studies [2, 31]. To build the non-flaky population, Rahman et al. started with projects evaluated by `FlakyCat` and extracted 179 403 candidate tests. After removing all tests that share a test class with a known flaky test, they ran each remaining candidate 100 times to confirm it was non-flaky. The resulting dataset contains 8574 tests, of which 280 are flaky (3.3% across the five flakiness categories `FlakeBench` retains from `FlakyCat`[2]) and 8294 are non-flaky, spanning 98 projects.[3] `FlakeBench` thus pairs report/category-mined flaky labels with rerun-confirmed non-flaky labels.

The contrast between these strategies motivates our own `C-IDoFT`: Rather than inheriting `IDoFT`'s fix-commit non-flaky labels, we extend `IDoFT` by deriving non-flaky tests from repeated executions of the modules that contain flaky tests, similar to `FlakeBench`'s rerun-confirmed non-flaky construction [17] (Section III). `FlakeBench` and `C-IDoFT` share the rerun-confirmed non-flaky rule, but differ in what they hold fixed, and the difference determines what each can establish. `FlakeBench` substitutes the flaky population (drawing from `FlakyCat` and discarding the Network, Randomness, and Resource Leak categories), the projects, and the non-flaky construction together, producing a separate benchmark. A separate benchmark confounds the question we set out to answer: Lower performance could mean its tests are harder to classify or differently distributed. To attribute a change in performance to the labeling rather than to the dataset, the flaky tests must be held fixed while only the non-flaky construction varies. `C-IDoFT` is built for exactly this purpose: It uses the same `IDoFT` benchmark that the detectors we reproduce were evaluated on, restricts its flaky tests to developer-confirmed reports, and changes only how the non-flaky class is constructed.

C. Reproduced Detectors

We reproduce three existing approaches that use LLMs for code-based flaky test detection [16, 18, 21]. There are other studies on code-based flaky test detection, such as vocabulary-based token models [24], combinations of expert features with CodeBERT embeddings [10], and classification from statically extracted features [14]. We focused on `Flakify`, `FlakyQ`, and `FlakyXBert` because they are LLM-based, were published at major SE venues (TSE and ICST), and provide replication packages.

Flakify uses a fine-tuned version of *CodeBERT*, a pre-trained language model with 125 million parameters [16, 32]. Under cross-validation, `Flakify` reports an almost perfect F1 score of 0.98 on `IDoFT` but only 0.79 on the rerun-labeled `FlakeFlagger` dataset; under per-project validation, the gap persists (0.89 on `IDoFT` versus 0.73 on `FlakeFlagger`). In a follow-up study, Rahman et al. found that `Flakify`'s implementation suffered from data leakage in its cross-validation setup, where a single CodeBERT instance was reused across folds rather than reloaded, inflating its reported F1 score [17]. We found that the same data leakage also affected `Flakify`'s published leave-one-project-out evaluation: Projects were evaluated sequentially, and the model was not reinitialized between held-out projects, so later evaluations could reuse weights already updated on earlier runs. The results without this leakage remained unknown. We fill this gap by replicating `Flakify` with pipelines that reload the model for each fold or held-out project.

FlakyQ is a quantized version of `Flakify` that improves inference efficiency. `FlakyQ` achieved an F1 score of up to 0.94 on `IDoFT` while reducing prediction time and RAM usage by 25% and 48% [21]. Its evaluation uses the `IDoFT` variant (3813 tests) and the `FlakyCat` categorized flaky tests, not the rerun-labeled `FlakeFlagger` set. As `FlakyQ` reuses `Flakify`'s training and evaluation pipeline, adding quantization for inference, it suffers from the same data leakage.

FlakyXBert uses few-shot learning (FSL) with a Siamese network to train a CodeBERT-based flakiness detector [18]. More and Bradbury concluded that FSL yields a competitive

[1] `Flakify`'s original evaluation used 3862 tests (3268 flaky, 594 non-flaky), consistent between its paper and its replication package. The 3813-test variant is the one shipped in `FlakyQ`'s replication package, which we and the reproduced studies use. Its label column contains 3226 flaky and 587 non-flaky tests rather than the reported 3195 and 618.

[2] Rahman et al. state 3.4%; 280 of 8574 tests is 3.3%, whereas 3.4% corresponds to the ratio of flaky to non-flaky tests (280/8294).

[3] Rahman et al. report 97 projects; the shipped dataset spans 98 projects, one of which contributes only non-flaky tests.

F1 score compared with fine-tuning, while requiring less data and fewer computational resources, and thus may be better suited for settings where the model must adapt rapidly to changes in the code base without extensive retraining. `FlakyXBert` is evaluated on the `IDoFT` variant and the `FlakyCat` dataset.

Of the three reproduced approaches, only `Flakify` reports results on the rerun-labeled `FlakeFlagger` dataset, and that is precisely the dataset on which it scores lowest (0.79 versus 0.98 on `IDoFT`). `FlakyLens` [17][4], a focal-loss CodeBERT flakiness detector, is introduced together with `FlakeBench` and evaluated on it; neither `FlakyQ`, `FlakyXBert`, nor `FlakyLens` reports results on `FlakeFlagger`'s rerun-based labels. The benchmarks the field continued to use are those whose non-flaky labels do not come from deep reruns (the `IDoFT` variant's fix-commit labels or `FlakeBench`'s 100-run confirmation), while the dataset whose non-flaky labels come purely from deep reruns is the one on which the strongest reported detector already struggled. We add rerun-confirmed non-flaky tests to the `IDoFT` benchmark to see whether this influences the performance reported in prior studies.

D. Flaws in Prior Work and the Gap We Address

Prior work has identified a consistent set of flaws in code-based flaky test detection, but reports each as an isolated defect. The trained models do not generalize, degrading on newer project revisions [12], on unseen projects [25], and on industrial data, where one detector was evaluated but never deployed [13]. Some reported gains are also experimental artifacts, such as the cross-validation data leakage Rahman et al. traced through a prior detector [17]. A deeper problem is that the test code may not carry the signal at all: The widely used `IDoFT` benchmark pairs each flaky test with a near-identical developer-fixed version, so a model can separate the classes from superficial similarity rather than from flakiness. Even human experts often do not consider themselves able to label a test from its code alone [19].

Each flaw has been treated as something to patch, a leak to fix or a benchmark to clean, leaving open whether the task itself is well-posed. Answering that requires a controlled counterfactual without the shortcut, which is a central contribution of our study. We construct `C-IDoFT` for this purpose (Section II-B), removing the fix-commit shortcut while restricting the flaky tests to a developer-confirmed subset of the same benchmark. If code-based detectors still fail on `C-IDoFT`, the limitation lies in the task rather than in flaws of a particular dataset. `C-IDoFT` thus serves as a control, not as a benchmark we recommend adopting. We then construct `FlakeCI`, a dataset of flaky E2E failures mined from CI logs, to examine whether the evidence for diagnosing a flaky failure lies in the test code, in the CI log, or beyond both. Together these datasets let us reproduce three published LLM-based detectors under a single rigorous protocol, project-disjoint wherever the design permits it, and ask whether the test code contains the discriminating signal at all, rather than only whether their reported F1 scores can be reproduced.

[4]Our replication package additionally includes `FlakyLens+` evaluations (Section III) on `C-IDoFT`, `FlakeFlagger`, and `FlakeBench`, provided for completeness, together with four-class category classification runs on `C-IDoFT`, which this paper does not analyze because they address a different task: predicting a test's flakiness category rather than the binary detection our research questions study; `FlakyLens` is not among the three detectors we reproduce.

III. Study Design

In the following, we introduce our research questions and the methodology to answer them.

RQ1: *To what degree can we replicate the performance of existing code-based flaky test detectors?*

Motivation. Despite recurring promising results reported by studies on code-based flaky test detection, recent work cast doubts on the generalizability of the trained models and their applicability in practice [17, 19]. While the original vocabulary-based approach has been replicated on newer revisions, unseen projects, and industrial data (Section II), there is no evidence on whether the identified generalizability issues also apply to code-based detectors that use LLMs.

Approach. We adhere to ACM's definition for artifact review and aim to first reproduce the results of the given studies using the author-supplied artifacts in the replication packages of the respective studies [33]. We focus our reproduction on the experiments related to flakiness detection. For a sound comparison, we use the same performance metrics as the original papers, namely the weighted F1 score and the flaky-class F1 score. Given the precision and recall for a class c, the F1 score for c is the harmonic mean $F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$. The flaky-class F1 score $F1_f$ is $F1_c$ evaluated for the flaky class. The weighted F1 score $F1_w$ is the weighted average over a set of classes C, where each class $c \in C$ has support n_c and $N = \sum_{c \in C} n_c$, such that $F1_w = \sum_{c \in C} \frac{n_c}{N} \cdot F1_c$. $F1_w$ is dominated by whichever class holds the majority: the flaky class on the `IDoFT` variant used in **RQ1**, whose non-flaky tests are fix-commit versions and where 3195 of 3813 tests are flaky, and the non-flaky class on the rerun-labeled datasets, where only 2–4% of tests are flaky. On the latter, $F1_w$ can approach 1 even if the detector detects almost no flaky tests. We therefore report the flaky-class metrics as the more informative measures of detection and compare every result against a majority-class baseline, which always predicts flaky or always predicts non-flaky, depending on the dataset's majority class. On the flaky-class metrics, we additionally report the always-flaky predictor, whose flaky-class F1 score is the maximum any input-agnostic predictor can attain ($F1_f = 2p/(1+p)$ at recall 1, for flaky share p). Cross-validation results are reported as the fold mean, with $\pm$ denoting the fold-to-fold standard deviation. In the project-disjoint setting, no project contributes tests to both the training and the test set of a given fold, so the reported performance reflects generalization to unseen projects. After reproducing the results using the original pipelines, we replicated the proposed approaches in the project-disjoint setting defined above. This is similar to the setup of Camara et al. [25], who

found that this setting leads to a performance drop for the lexical code-based models.

In addition to the quantitative results from the evaluation pipelines, we performed a qualitative analysis of the datasets to understand the characteristics that LLMs can use to distinguish flaky from non-flaky tests given these benchmarks.

> **RQ2**: *What is the performance of CodeBERT-based detectors on a controlled-counterfactual dataset that removes the fix-commit labeling shortcut?*

Motivation. Previous work pointed out that the IDoFT benchmark, which Flakify [16], FlakyXBert [18], and FlakyQ [21] used for evaluation, exhibits spurious features that can be exploited by a trained model to distinguish non-flaky and flaky tests [19]. The authors traced this to the dataset's construction: Non-flaky samples are the fixed versions of previously flaky tests, so a model can separate the two classes from characteristics the fix introduces, such as the shared fix code itself, rather than from any signal of flakiness. FlakeBench responded to this problem with a new dataset whose flaky tests come from a different source and different projects. C-IDoFT instead keeps a developer-confirmed subset of IDoFT's flaky tests and changes only the non-flaky labeling, so that a change in performance is attributable to the labeling and not to the different set of tests a new dataset would introduce (Section II-B).

Approach. To mitigate the fix-commit shortcut, we first curated a new dataset from $n = 500$ repeated test executions across the projects in the IDoFT benchmark, a common approach for labeling tests in flakiness datasets [15, 34]. The run budget of $n = 500$ was chosen to balance statistical confidence with the computational cost of module-level execution. Given a test with an independent per-run failure probability p, the probability of observing at least one failure in n runs is $1 - (1 - p)^n$. With $n = 500$, a test with a failure probability of 1% has a 99.3% chance of failing at least once [35]. After repeatedly executing the tests and labeling them, we fine-tuned our two CodeBERT reimplementations, Flakify+ (class-weighted cross-entropy, following Flakify's training setup) and FlakyLens+ (focal loss, following FlakyLens), on C-IDoFT and evaluated their performance. For evaluation, we report the flaky-class precision, recall, and F1 score as our primary metrics, and report the weighted F1 score only for comparison with the published results, as motivated above. We additionally compare against a majority-class (always-non-flaky) baseline. As in **RQ1**, we evaluated in a project-disjoint setting, holding out entire projects from training. Table I summarizes the sizes of the datasets used to answer **RQ1** and **RQ2**.

TABLE I
THE THREE DATASETS USED TO ANSWER **RQ1** AND **RQ2**, WITH THEIR PROJECT AND FLAKY/NON-FLAKY TEST COUNTS. FOR FLAKEFLAGGER, WE LIST THE 20 858 NON-FLAKY TESTS THAT ENTER OUR PROJECT-DISJOINT EVALUATION; THE ORIGINAL PAPER REPORTS 20 859 BEFORE REMOVING ONE TEST WITHOUT A PROJECT LABEL [16]. THE IDOFT ROW LISTS THE SPLIT REPORTED BY FLAKYQ (SEE FOOTNOTE 1).

Dataset	Projects	Flaky	Non-flaky
FlakeFlagger	23	802	20 858
C-IDoFT	57	1319	53 149
IDoFT	126	3195	618

> **RQ3**: *To what degree can we diagnose flaky failures in E2E tests given the test code and CI logs?*

Motivation. Flakiness is especially costly in slow and expensive E2E test suites, where extensive reruns are impractical, and failures are hard to diagnose [36, 37]. In JavaScript, the language of the E2E suites we study, flaky tests are dominated by concurrency, async-wait, OS, and network causes [38]. Yet the projects that make up existing flaky-test benchmarks are, by construction, those that can be built and executed quickly enough to be rerun at scale, often with build times of only a few minutes, which biases the field toward rather simple unit tests and away from the complex systems where flakiness is most problematic and expensive [4]. Project selection thus matters as much as test selection. If flakiness in these E2E settings is driven by factors not observable in the test source code, then code-based detection is the wrong approach precisely where it matters most. This concern is consistent with recent evidence that even human experts often do not consider flakiness identifiable from the test code alone, depending on the type of flakiness [19].

The flakiness of E2E tests has different causes from those of the small unit tests of **RQ1** and **RQ2**, so simply reporting that a detector scores poorly on them would prove little, since the drop could reflect a harder and different domain rather than the task framing, the same confounding factor we avoid in **RQ2**. We therefore do not evaluate code-based detection performance on these tests. Instead, we manually assess whether the evidence explaining a flaky E2E failure is present in the test code and CI logs.

Approach. In contrast to prior work, which identified flaky E2E tests in open-source projects based on flakiness-related keywords in commits and closed issues [39], or detected build-level flakiness from restarted CI builds whose outcome changed on the same code [40], we constructed FlakeCI by mining the GitHub Actions CI logs of open-source Cypress and Playwright projects, identifying E2E tests that both pass and fail on the same commit within the same workflow and job context, so that outcome differences produced by unrelated configuration changes, such as different runner images or environment variables, are excluded. One author assigned each of the 86 confirmed flaky tests a primary cause category from the test title, test code, and the matched failing and passing log evidence. The annotating author discussed the category set and the annotation procedure with a second author and assigned a test to the *Unknown* category whenever the evidence did not support a cause without speculation. We then quantified the share of flaky failures whose diagnosis depends on execution evidence beyond the test code and CI logs.

IV. Results

All experiments were executed on an Apple Mac Studio with an Apple M3 Ultra chip (32-core CPU, 80-core GPU) and 512 GB of unified memory.

A. RQ1: Replication

With the first research question, we investigated whether the previous results of existing code-based detectors could be validated in a more rigorous evaluation. As described in Section III, we started by reproducing the existing studies given the author-supplied replication packages. We report the results of reproducing `Flakify` and `FlakyQ` on the `IDoFT` benchmark in Table II. Run as shipped, the replication packages yielded results similar to the values reported in the paper, with differences of at most 0.02. After we fixed the data leakage, the weighted F1 score dropped from 0.95 to 0.88 for both `Flakify` and `FlakyQ`. The per-fold flaky-class F1 score showed the same drop (`Flakify` 0.975 ± 0.014 to 0.919 ± 0.026; `FlakyQ` 0.969 ± 0.019 to 0.925 ± 0.025) and remained high, as the flaky class is the majority on this dataset. Under project-disjoint evaluation, the weighted F1 score fell further to 0.80, matching the always-flaky baseline of 0.80 on the same held-out set. As mentioned in Section II, in addition to `IDoFT`, `Flakify` was evaluated on `FlakeFlagger`. We report the results before and after fixing the data leakage in Table IV. The `FlakeFlagger` results repeated the `IDoFT` pattern: The flaky-class F1 score collapsed after fixing the leakage (from 0.79 to 0.64) and again under project-disjoint evaluation (to 0.07). Under the same per-project aggregation, the always-flaky baseline reached 0.14, so the detector did not exceed it.

For `FlakyXBert`, we noticed a mismatch between the projects reported in the paper and those analyzed in the notebooks. Therefore, we report the per-project results of `FlakyXBert`'s reproduction in Table III. Most of the F1 scores showed only minor deviations. However, the outputs shipped in four of the notebooks stem from another project's run, leading to an inflated performance estimate in all cases.[5] With the notebooks rerun on the correct data, the F1 score dropped from 0.94, which closely resembles the value reported in the paper, to 0.83.

Since `FlakyXBert`'s authors did not include the results from all projects in their paper, restricting the aggregate to the projects reported there yielded an F1 score of 0.88, while the paper reported 0.95. After we notified them, the authors acknowledged this mistake in the paper and confirmed the F1 score reported in the notebooks. Unlike `Flakify` and `FlakyQ`, `FlakyXBert` cannot be replicated under our project-disjoint protocol: Its few-shot design requires labeled examples from the target project, so its per-project scores measure within-project classification and leave cross-project generalization untested.

[5]In all four cases, the notebook's code names the correct project, but its saved outputs, including the per-sample predictions, are identical to those of another project's notebook: `commons-lang` and `flink` match `spring-data-r2dbc`, `fastjson` matches `admiral`, and `graylog2-server` matches `mockserver`. Whether the wrong data was loaded at run time or outputs from another run were retained cannot be determined from the released artifact.

TABLE II
Average weighted F1 score in 10-fold cross-validation on the `IDoFT` dataset before and after fixing the data leakage. Paper: the studies' reported values. Prefix and Postfix: our reproduction before and after this fix (model reloaded per fold). Disjoint: the project-disjoint evaluation. Baseline: the always-flaky weighted F1 score on the full `IDoFT` (3813 tests, labeled as shipped; see footnote 1); on Disjoint's held-out 2105 tests, this baseline is 0.80.

Classifier	Paper	Prefix	Postfix	Disjoint	Baseline
`Flakify`	0.94	0.95 ± 0.02	0.88 ± 0.03	0.80	0.78
`FlakyQ`	0.93	0.95 ± 0.03	0.88 ± 0.03	0.80	0.78

Beyond the four notebooks with another project's outputs, our qualitative analysis revealed a preprocessing shortcut. The `Flakify` pipeline keeps only statements associated with targeted test smells, so tests without such smells collapse to empty method bodies. In `adyen-java-api-library`, 76 of 89 tests in the shipped CSV have such empty method bodies; 36 of them are empty because their rows are corrupted, not because of the preprocessing. We repaired these rows from the pipeline's source dataset to check whether the corruption explained `FlakyXBert`'s low score on this project: The per-project F1 score rose from 0.31 to 0.94. The gain, however, rests on the shortcut, not on flakiness detection. After the repair, 32 of 35 non-flaky training examples have empty method bodies and none of the 36 flaky ones do, so separating empty from non-empty methods suffices.

Answer RQ1: The results of `Flakify` and `FlakyQ` were reproducible within 0.02 with the author-supplied replication packages. After fixing the data leakage, the weighted F1 score dropped from 0.95 to 0.88 for both approaches, and under project-disjoint evaluation neither meaningfully exceeded the always-flaky baseline: 0.80 versus 0.80 in the weighted F1 score on the held-out `IDoFT` projects, and 0.07 versus 0.14 in the per-project flaky-class F1 score on `FlakeFlagger`. For `FlakyXBert`, four notebooks ship outputs from another project's run, inflating the F1 score; after correction, its weighted F1 score dropped from 0.94 to 0.83, remaining above the always-flaky baseline of 0.66, unlike `Flakify` and `FlakyQ`. Its few-shot design trains on labeled tests from the project it classifies, so 0.83 is not a cross-project result.

B. RQ2: A Controlled Counterfactual (`C-IDoFT`)

With the second research question, we aimed to find out whether our observations from **RQ1** hold on a controlled-counterfactual dataset that does not use fixed flaky tests as non-flaky samples. We began with the `IDoFT` dataset as

TABLE III

WEIGHTED F1 SCORE IN PER-PROJECT EVALUATION OF FLAKYXBERT ON THE IDOFT DATASET. SHIPPED: THE VALUES RENDERED IN THE REPLICATION PACKAGE'S NOTEBOOKS, NOT REPORTED IN THE PAPER. FOR PROJECTS MARKED WITH †, THE SHIPPED NOTEBOOKS CONTAIN OUTPUTS FROM ANOTHER PROJECT'S RUN RATHER THAN FROM THE PROJECT'S OWN DATA. PAPER: THE PAPER'S REPORTED VALUES. RERUN: EACH PROJECT'S NOTEBOOK RE-EXECUTED ON ITS OWN DATA. WEIGHTED AVERAGE: PER-PROJECT VALUES AGGREGATED BY SUPPORT.

Project	Rerun	Shipped	Paper	Always flaky
junit-quickcheck	0.92	0.94	0.94	0.36
dubbo	0.89	0.89	0.89	0.88
hadoop	0.95	0.95	0.95	0.97
nifi	0.92	0.92	0.92	0.93
admiral	0.91	0.91	0.91	0.95
fastjson†	0.55	0.91	0.91	0.43
spring-data-r2dbc	1.00	1.00	1.00	0.38
mockserver	1.00	1.00	1.00	0.67
commons-lang†	0.40	1.00	–	0.45
flink†	0.71	1.00	–	0.79
graylog2-server†	0.50	1.00	–	0.46
adyen-java-api-library	0.31	0.31	0.30	0.34
Weighted Average	**0.83**	**0.94**	**0.95**	**0.66**

TABLE IV

FLAKIFY ON FLAKEFLAGGER, REPORTED AS THE AVERAGED FLAKY-CLASS F1 SCORE FROM CROSS-VALIDATION, FOLLOWING THE ORIGINAL PAPER. THE PROJECT-DISJOINT ROWS AND THE ALWAYS-FLAKY BASELINE ARE PER-PROJECT MEANS.

Evaluation	Flaky-class F1 score
Paper Average	0.79
Corrected Average	0.64
Project-disjoint, per-project mean	0.07
Always flaky, per-project mean	0.14

used by Flakify. Flakify's authors used all IDoFT flaky tests, including those whose flakiness was reported but never confirmed by a developer. We restricted the flaky tests in C-IDoFT to developer-confirmed tests from IDoFT. We refer to a test as *developer-confirmed* when the resolution status of its linked pull request is one of *Accepted*, *DeveloperFixed*, or *InspiredAFix* (status values of IDoFT's PR tracking), indicating that the reported flakiness was confirmed by a developer rather than merely reported or automatically detected. We first kept projects with a publicly accessible, non-archived, non-fork GitHub repository. We then required at least 10 developer-confirmed flaky tests per project, which left 69 candidate projects, of which 57 could be built and executed at the commits where the flakiness had been reported. From these 57 projects, we executed the modules containing developer-confirmed flaky tests 500 times. We labeled a test as non-flaky when all 500 executions yielded consistent results.

As model input, we used the full test code, filtered to at most 1024 CodeBERT tokens, avoiding smell-based preprocessing and the empty method bodies it induces. The 1024-token cap doubles Flakify's 512-token input limit while remaining compatible with CodeBERT's 512-position maximum, which our trainer handles by averaging two 512-token chunks; the cap retains 97.3% of the extracted tests. We evaluated both variants with four project-disjoint folds, fine-tuning CodeBERT[6] for up to 20 epochs with early stopping (patience 5, batch size 4, learning rate 10^{-5}, seed 42). The training folds drew their non-flaky class from a source deduplicated by template body ($42\,527$ of $54\,468$ rows, retaining all 1319 flaky tests); all reported metrics were computed on the full held-out test folds.

Table V shows our project-disjoint results on C-IDoFT and FlakeBench. On C-IDoFT, both CodeBERT variants collapsed: Flakify+ reached a flaky-class F1 score of 0.035 (recall 0.021) and FlakyLens+ 0.070 (recall 0.120), so both missed almost every flaky test. Neither meaningfully exceeded the constant baselines. Flakify+ stayed below the always-flaky predictor's flaky-class F1 score (0.035 versus 0.054), and FlakyLens+ exceeded it by less than its own fold-to-fold variability (0.070 versus 0.054). The weighted F1 score of neither variant exceeded the always-non-flaky baseline at the reported precision (0.958 and 0.948 versus 0.958). The per-fold flaky-class F1 score varied as much as its mean, so the small gap between the two variants does not show that one is better than the other.

On FlakeBench, evaluated identically, the same variants recovered 261 of 263 flaky tests[7] (flaky-class F1 scores of 0.984 and 0.967), so the test code carries a separable signal for that curated set. In FlakeBench, most projects contribute exactly one flaky test (63% of projects; a median of one per project), so its project-disjoint success rests on a signal shared across projects, whereas C-IDoFT concentrates flaky tests by construction (a median of 13 per project). A plausible reason is selection: FlakeBench's flaky tests stem from FlakyCat [30], which assigned a root cause category only where the cause could be recognized from the developer's fix and commit message, and the retained categories, such as unordered collections and async waits, are types whose flakiness tends to be apparent in the test code. The flakiness tracked in IDoFT includes types that are not identifiable from the test code alone [19]. We therefore read the FlakeBench result as lexical separability of a curated benchmark, not as evidence that source-only detection generalizes to developer-confirmed flakiness with rerun-confirmed non-flaky labels. To complete the counterfactual square (the two labelings, fix-commit and rerun-confirmed, crossed with the two evaluation protocols), we additionally evaluated both variants on C-IDoFT under label-stratified random 10-fold cross-validation, the protocol class of the published evaluations. Under this protocol, Flakify+ reached a weighted F1 score of 0.989 with a flaky-class F1 score of 0.746 ± 0.030 (recall 0.641), and FlakyLens+ 0.983 with 0.686 ± 0.109 (recall 0.709). Table V reports these as the C-IDoFT (CV) rows. Both variants scored far above the constant baselines of the

[6]Checkpoint microsoft/codebert-base, https://huggingface.co/microsoft/codebert-base.

[7]The 280 flaky tests of FlakeBench (Section II-B) reduce to 263 under the $\leq$1024-token CodeBERT input filter we apply to match the C-IDoFT evaluation; the filtered set contains 263 flaky and 7985 non-flaky tests.

random folds (always-flaky flaky-class F1 score 0.047, always-non-flaky weighted F1 score 0.964) on the same data on which both variants had collapsed under project-disjoint evaluation: The protocol change alone moved the flaky-class F1 score from 0.035 to 0.746. Random cross-validation lets the model exploit within-project similarity, and this apparent detection ability does not transfer across projects.

To isolate the effect of the labeling alone, we also evaluated the fix-commit-labeled `IDoFT` dataset (3,671 of 3,813 tests after the token filter) under the identical conventions (full test code, at most 1024 tokens, same fold generator and seed): `Flakify+` reached a weighted F1 score of 0.89 with a flaky-class F1 score of 0.937 $\pm$ 0.016, and `FlakyLens+` 0.83 with 0.878 $\pm$ 0.047. `Flakify+`'s result lies in the published fix-commit range of 0.88–0.95, and on this flaky-majority dataset the always-flaky predictor alone attains a flaky-class F1 score of 0.916. We held the input conventions fixed rather than reproducing `Flakify`'s test-smell preprocessing: The smell detector requires complete Java test-class sources, which are not part of `C-IDoFT`'s model inputs, and it is Java-specific, whereas `C-IDoFT` also contains tests written in Kotlin and Groovy. Reproducing that preprocessing would add a dataset construction step with its own failure modes and, more importantly, weaken the experiment by reintroducing the empty method bodies it induces. Under project-disjoint folds, the same input stayed at its constant baseline: `Flakify+` reached a weighted F1 score of 0.79 and `FlakyLens+` 0.81 against an always-flaky fold mean of 0.78, with `Flakify+` predicting nearly every test flaky (flaky-class recall 0.996). We therefore compare the two labelings under one set of conventions. Table VII (Section V) assembles the four cells and their baselines.

Answer RQ2: Our controlled counterfactual `C-IDoFT` confirms the findings of **RQ1**: On flaky tests drawn from the same benchmark, once the non-flaky class is rebuilt from reruns, both CodeBERT variants detected almost no flaky tests and did not meaningfully exceed any constant baseline (flaky-class F1 scores of 0.035 and 0.070 versus the always-flaky predictor's 0.054; the weighted F1 score not above the always-non-flaky baseline). Under the published cross-validation protocol, the same variants scored far above these baselines on the same data (flaky-class F1 scores of 0.746 and 0.686). Project separation removes this apparent detection ability. The same variants succeeded on the curated `FlakeBench` (flaky-class F1 scores up to 0.984), so the test code carries a separable signal there but not for developer-confirmed flakiness with rerun-confirmed non-flaky labels.

C. RQ3: Diagnosing Flaky E2E Failures (`FlakeCI`)

With the third research question, we aimed to determine the extent to which flaky failures in E2E tests require signals beyond the test source code. For this question, we first curated a dataset by mining CI logs from GitHub projects. We started by identifying repositories containing GitHub Actions workflow files that signal Playwright or Cypress test execution, yielding 522 Playwright-enabled and 377 Cypress-enabled repositories. We then collected CI run data for the 90 days preceding the observation date, the maximum period for which GitHub Actions retains execution logs. The collection windows fell between January and April 2026. We labeled a test as flaky according to the criterion defined in Section III, requiring explicit test-level evidence from the CI logs for both the failing and passing observations. The resulting dataset contains 86 flaky tests from 26 repositories: 74 Cypress tests from 22 repositories and 12 Playwright tests from 4 repositories.

TABLE V
RESULTS ON C-IDOFT (THE CONTROLLED COUNTERFACTUAL) AND FLAKEBENCH (A CURATED BENCHMARK). $F1_f$ AND RECALL$_f$ ARE FOR THE FLAKY CLASS; $F1_w$ IS THE WEIGHTED F1 SCORE. THE CONSTANT ROWS (SECTION III) ARE COMPUTED PER HELD-OUT TEST FOLD FROM ITS FLAKY SHARE; ALWAYS NON-FLAKY NEVER PREDICTS THE FLAKY CLASS, SO ITS $F1_f$ IS UNDEFINED (–). ALL ROWS ARE PROJECT-DISJOINT EXCEPT THE TWO C-IDOFT (CV) ROWS, WHICH REPORT RANDOM CROSS-VALIDATION (CV): THE (C-IDOFT, CV) CELL OF TABLE VII.

Dataset	Approach	$F1_f$	Recall$_f$	$F1_w$
C-IDoFT	Flakify+	0.035 $\pm$ 0.058	0.021	0.958
C-IDoFT	FlakyLens+	0.070 $\pm$ 0.078	0.120	0.948
C-IDoFT	Always flaky	0.054 $\pm$ 0.020	1.000	0.002
C-IDoFT	Always non-flaky	–	0.000	0.958
FlakeBench	Flakify+	0.984 $\pm$ 0.028	0.992	0.999
FlakeBench	FlakyLens+	0.967 $\pm$ 0.029	0.992	0.998
FlakeBench	Always flaky	0.063 $\pm$ 0.009	1.000	0.002
FlakeBench	Always non-flaky	–	0.000	0.951
C-IDoFT (CV)	Flakify+	0.746 $\pm$ 0.030	0.641	0.989
C-IDoFT (CV)	FlakyLens+	0.686 $\pm$ 0.109	0.709	0.983

Table VI summarizes the root cause categories assigned to the 86 flaky tests. For 42% of them (36 of 86), we could attribute a cause from the test code and CI log. For the remaining 58% (50 of 86), we could not attribute a cause without speculation, so these were recorded as *Unknown*. Among the attributable cases, Network or external service instability was the largest category (27% of all 86 tests). Two projects, `osmosis-frontend` and `infinispan-console`, account for 15 of the 23 Network cases.

Three cases show what the CI run captures and the test code does not. In `osmosis-frontend`, a swap test (*User should be able to select stATOM/USDC*) failed with a 90-second timeout while waiting for token price data after opening the live application, failing 44 times and passing 3 on the same commit. In `bratislava.sk`, an E2E test of the site's official-board page failed because `cy.visit` returned HTTP 503 from the live site. In `processmaker/screen-builder`, a date-picker test failed on an invalid WebSocket URL `ws://localhost:undefined/` formed during application setup. In all three cases, the failure's cause was a runtime condition (a slow price feed, an unavailable site, an undefined port) that the test code does not show.

TABLE VI
ROOT CAUSE CATEGORIES ASSIGNED TO FLAKECI'S 86 FLAKY TESTS (74 CYPRESS + 12 PLAYWRIGHT FROM 22+4 REPOSITORIES).

Cause category	Playwright	Cypress	Both	
			$\sum$	%
Unknown	3	47	50	58
Network / external service	9	14	23	27
Environment / config	0	7	7	8
Resource / state cleanup	0	5	5	6
Async wait	0	1	1	1
Total	**12**	**74**	**86**	**100**

Answer RQ3: The test code and CI log sufficed to attribute a cause for 42% of the flaky tests (36 of 86). Diagnosing the remaining 58% would require execution evidence beyond these two artifacts. Network or external service instability was the largest attributable category (27% of all 86 tests).

V. DISCUSSION

On the replication results. The results of our replication indicate that previous results on the IDoFT benchmark are caused by the evaluation protocol and metrics used, the lack of proper baselines, and the simple syntactical patterns that sufficed to distinguish flaky from non-flaky tests in that data split, without any signal of flakiness itself. These patterns include preprocessing artifacts such as class-correlated empty test bodies. Additionally, we found the results of Flakify on FlakeFlagger to be mostly explained by the data leakage problem that Rahman et al. [17] already described. Our reproduction after fixing the data leakage issue showed that the LLM performs only slightly better than our simple baseline. This finding is in line with prior work stating that the test code alone may not suffice as a signal to distinguish flaky from non-flaky tests in a meaningful scenario [19].

On the controlled counterfactual C-IDoFT. The results on the controlled counterfactual, without shortcuts to distinguish flaky from non-flaky tests, support this claim. Because C-IDoFT changes only the non-flaky labeling of the benchmark prior work used, this result cannot be attributed to a harder or unrelated dataset. We argue that our results align with the replication studies [12, 13, 25] of the original lexical approach by Pinto et al. [11], which have already shown that code-based detectors lack generalizability when evaluated on real-world data. Our results suggest that these weaknesses cannot be mitigated by LLMs, as they are rooted in the task's framing. This holds beyond the CodeBERT-based detectors we evaluated: Prompting GPT-4o, GPT-OSS, and Qwen3-Coder classified flakiness from the test code only marginally better than random guessing [19]. As pointed out by prior work [37], when the number of repeated executions of a test goes toward infinity, every test can eventually yield a failing execution. For example, even central processing units (CPUs) have been shown to be susceptible to "silent data corruptions", which propagate to the application level and may cause flaky failures [41]. Therefore, we argue that flakiness is not a static property that a test either has or does not have. It typically emerges from multiple contributing factors during runtime.

On the analysis of FlakeCI. Our analysis of the newly curated FlakeCI dataset showed that diagnosing most E2E flaky tests required execution evidence beyond the test code and CI logs. For most of the tests (58%), the cause could not be attributed from the test code or the CI logs alone. This finding indicates that flakiness requires more in-depth debugging and observability tooling to pinpoint the root causes of flaky failures and inform decisions on whether a failure is caused by an actual defect or by flakiness.

Implications for evaluating detectors. Our reproduction points to a small set of reporting practices that would have exposed the fragilities we found. Detector studies should report the flaky-class precision, recall, and F1 score as primary metrics, together with their fold-to-fold variability, since a high weighted F1 score can coexist with near-zero flaky-class recall on imbalanced data. Such studies should evaluate their models under project-disjoint splits and against a majority-class baseline. A score that merely matches this baseline should not be read as detection. Input-aware trivial baselines, such as a linear model over token counts, and threshold-free metrics such as the area under the precision-recall curve [42] would further tighten the attribution of high scores to labeling shortcuts rather than to detection ability. We leave both to future work. Table VII assembles the counterfactual square from **RQ1** and **RQ2**: both non-flaky labelings evaluated under each protocol, each cell against the strongest constant baseline of its dataset. We refer to a cell by its (labels, protocol) pair, such as (IDoFT, CV). The two datasets differ in flaky share, so scores compare against their own baseline, not across rows. Under project-disjoint evaluation, neither labeling exceeds its baseline. Under random cross-validation, both do: Even with rerun-confirmed labels, the flaky-class F1 score rises from 0.035 in (C-IDoFT, Disjoint) to 0.746 in (C-IDoFT, CV), on identical data, once projects recur across folds. The cross-validation protocol thus produces scores above the constant baselines even without the labeling shortcut, while the shortcut remains visible within the protocol: In (IDoFT, CV), the fix-commit labels reproduce the published range. Whatever signal the models exploit under cross-validation, with either labeling, does not transfer across projects. Non-flaky labels should come from repeated executions or CI history, with the observation protocol reported, rather than from fix-commit versions of flaky tests, which encode a near-duplicate shortcut. Preprocessing should likewise be checked for class-correlated artifacts such as empty test bodies before model scores are reported. Finally, a gain that appears only on a benchmark whose classes are lexically separable, such as FlakeBench, should be reported as such, not as evidence of general flaky-test detection. Our replication package provides the project-disjoint pipelines, the C-IDoFT curation, and the FlakeCI evidence bundles to support these practices.

TABLE VII

NON-FLAKY COUNTERFACTUAL: WEIGHTED F1 SCORES OF THE DETECTORS UNDER THE FIX-COMMIT (IDoFT) AND RERUN-CONFIRMED (C−IDoFT) NON-FLAKY LABELINGS AND BOTH PROTOCOLS, RANDOM CROSS-VALIDATION (CV) AND PROJECT-DISJOINT FOLDS (DISJOINT), TOGETHER WITH EACH GROUP'S CONSTANT BASELINE AND Δ (BEST VALUE MINUS BASELINE). ROWS NAME THE DETECTOR AND ITS INPUT CONVENTIONS: ORIGINAL (TEST-SMELL PREPROCESSING, ≤512 TOKENS) OR EXTENDED (FULL TEST CODE, ≤1024 TOKENS); FLAKIFY+ AND FLAKYLENS+ ARE OUR REIMPLEMENTATIONS (SECTION III). ORIGINAL-ROW VALUES ARE OUR REPRODUCTIONS, WITH * MARKING THE DATA-LEAKAGE FIX AND THE PUBLISHED SCORE IN PARENTHESES.

	CV	**Disjoint**
Fix-commit (IDoFT)		
Flakify (original)	0.95 (0.94), 0.88*	0.80*
FlakyQ (original)	0.95 (0.93), 0.88*	0.80*
Flakify+ (extended)	0.89	0.79
FlakyLens+ (extended)	0.83	0.81
baseline (always flaky)	*0.78*	*0.80*
Δ (best − baseline)	**+0.18**	**+0.01**
Rerun-confirmed (C−IDoFT)		
Flakify+ (extended)	0.989	0.958
FlakyLens+ (extended)	0.983	0.948
baseline (always non-flaky)	*0.964*	*0.958*
Δ (best − baseline)	**+0.025**	**+0.000**

Future work. Our results reframe flaky-test prediction as a question about executions rather than test code, and the instruments we release make that question actionable. We see three directions for the community, each grounded in a finding or an artifact of this study.

Failure-level classification. Instead of labeling a test, the field should move to classifying whether an individual observed *failure* is flaky from its execution evidence, such as logs, stack traces, and CI context. FlakeCI offers an initial labeled corpus, and because our same-commit mining reads existing CI logs rather than re-running suites, it reaches the slow, complex systems that rerun-based labeling cannot, countering rerun-based labeling's bias toward projects that are easy to build and re-execute. The open questions are which runtime features separate flaky from genuine failures and whether they transfer across projects, languages, and frameworks.

Environment-conditioned failure probability. The binary test-level label should be replaced by a calibrated probability that a test fails given its execution environment, including the runner, operating system, browser, and network conditions. Such an estimate turns rerun budgeting into a decision under uncertainty, where how often to re-execute a failing test follows from its estimated failure probability and the asymmetric costs of a blocked pipeline against a shipped defect. This ties prediction to the compute cost that motivates flakiness handling, and measuring how much failure-level triage lowers that cost in industrial pipelines is itself an open question.

Language models that drive interventions. Our results discourage static LLM detection, but not the use of language models themselves. The evidence we find decisive, namely logs, stack traces, and CI context, is exactly the unstructured artifact such models read well. We see a promising direction in agents that move from reading to experimenting. Given a failing run, such an agent hypothesizes a cause and then perturbs the execution environment to test it, for example by injecting network latency or loss, adding CPU or memory pressure, shifting the system clock, or reordering tests, while observing which perturbations change the outcome. Terragni et al. explored such non-deterministic dimensions by re-executing flaky tests in dedicated containers [43]; the agent would select the perturbations from the failure evidence rather than enumerating them. This places the model inside the execution loop that static detection lacked, using it to characterize a test's failure distribution rather than to guess a label from source code.

VI. THREATS TO VALIDITY

We address threats to our study's internal, external, conclusion, and construct validity.

A. Internal Validity

We describe the degree to which we can rule out alternative explanations for our results [44].

On the reproduction of existing studies. We relied on the author-supplied replication packages when reproducing the existing pipelines of Flakify, FlakyQ, and FlakyXBert. Executing them locally required minor source code adjustments, such as updating paths and dependencies. Furthermore, unlike the authors of the studies, we did not run the experiments on CUDA devices but on a Mac Studio with Metal Performance Shaders (MPS), which might explain slight variations in the results of our experiments. Two observations limit the impact of this threat: The environment-induced variation stayed within 0.02 in our reproductions, an order of magnitude smaller than the effects our conclusions rest on, and the qualitative analysis of the underlying datasets ties the large drops after the leakage fix to dataset properties rather than to the execution environment.

On repeated executions for non-flaky labels. We conducted all repeated executions on a single CI configuration. Thus, flakiness triggered only under different OS versions, locale settings, or resource pressure [45] would be missed, potentially leading to tests being mislabeled as non-flaky. This limitation is inherent to rerun-based labeling rather than specific to our setup, as even FlakeFlagger's 10 000 reruns showed (Section II-B): Rarely failing tests lead to underestimating the number of flaky tests [34], and the executions needed to expose low-probability failures have been quantified [35]. For a test with a per-run failure probability of 1%, our 500-run budget observes at least one failure with a probability of 99.3%. Rarer failures are more likely to go unobserved and be mislabeled as non-flaky. A model that correctly flags such a test is then charged with a false positive, which lowers the measured flaky-class precision and F1 score. The flaky and non-flaky classes also carry labels from different oracles: developer confirmation yields high-precision flaky labels, and repeated execution is the established method for confirming non-flaky labels. The near-zero flaky-class recall

is computed over the developer-confirmed flaky tests, so mislabeled non-flaky tests do not enter that measurement, and our budget exceeds the 100 reruns common in prior curation [11, 17].

On the C-IDoFT construction. Relative to IDoFT, C-IDoFT narrows the flaky class to the developer-confirmed tests of the 57 buildable projects (1319 of 3195 tests) and shifts the flaky share from 84% to 2.4%. The collapse we report on C-IDoFT therefore reflects the labeling change together with these population shifts and the protocol change, which the counterfactual square separates (Table VII). On identical data and conventions, the protocol change alone moves the flaky-class F1 score from 0.746 in (C-IDoFT, CV) to 0.035 in (C-IDoFT, Disjoint), so we attribute the published scores to the labeling shortcut and the cross-validation protocol jointly rather than to the labeling alone. The $\leq$1024-token input filter (Section III) removes only 1536 of the 56 004 extracted tests (2.7%); the **RQ2** comparison applies the same filter to FlakeBench, so the performance difference between the two datasets cannot stem from the filter. The retained flaky tests, however, are the developer-confirmed subset, the highest-precision positive labels in IDoFT, which should make detection easier rather than harder. The 2.4% flaky share matches the 2–4% prevalence of the rerun-labeled datasets; a detector deployed on a full test suite would face a similarly small flaky minority.

On the manual root cause labeling. A single author categorized the 86 flaky tests in FlakeCI, which risks subjective assignments. We mitigated this by defining the category set and procedure with a second author and by anchoring every assignment to the matched failing and passing log evidence recorded in our replication package. When the evidence did not identify a cause, we assigned *Unknown* rather than speculate, so our reported 42% is a lower bound. We do not claim the remaining failures are undiagnosable from the code and logs, only that our manual analysis did not find a cause there. A same-commit pass/fail pair could still reflect deliberately different configurations rather than non-determinism, which our same-workflow-and-job criterion reduces (Section III). For FlakeCI's purpose here, determining whether failures can be diagnosed from the test code and CI logs, a single expert coder over 86 tests is adequate; a benchmark for training would require multiple independent coders and broader coverage.

B. External Validity

We describe the generalizability of our results [46].

On the benchmarks for code-based detection. The datasets used to answer **RQ1** and **RQ2** focus on Java projects using the Maven or Gradle ecosystem. Flakiness patterns in other ecosystems may differ in distribution and root causes, which would limit cross-language generalization. The detector results are therefore established only for unit tests in JVM languages, nearly all of them Java. FlakeCI covers JavaScript/TypeScript E2E tests, but we use it only for the diagnosis question of **RQ3**, not to evaluate the detectors. C-IDoFT contains only projects we could build and re-execute 500 times, so the **RQ2** findings come from projects that are comparatively easy to build and test. This selection works in the detectors' favor: The collapse appears on exactly the projects that are easiest to build, execute, and analyze. The restriction to Java projects with Maven or Gradle matches the setting in which the reproduced detectors and their benchmarks were developed, so it does not disadvantage them either.

On the curation of FlakeCI. We curated the FlakeCI dataset from a 90-day CI mining window, the maximum log retention of GitHub Actions. Thus, the dataset may capture seasonal patterns of that period rather than general problems. Furthermore, we focused on GitHub Actions runners, which might not generalize to other CI environments. The corpus is also small and concentrated: 86 tests from 26 repositories, 74 of them Cypress, and 15 of the 23 Network cases stem from two projects, so the reported category shares are indicative rather than representative. For its purpose here, probing whether causes can be attributed from the test code and CI logs, verified same-commit pass/fail evidence matters more than breadth; each of the 86 tests has both a failing and a passing run on the same commit.

C. Conclusion Validity

We describe the degree to which our data support the conclusions we draw [47].

Where our design yields paired observations, we tested the direction of each comparison with an exact two-sided sign test [47]. Fixing the data leakage lowered the weighted F1 score in 9 of the 10 cross-validation folds for both detectors (p = 0.02 each), and on FlakeFlagger the always-flaky baseline exceeded the classifier's per-project flaky-class F1 score in 19 of 23 projects (p = 0.003). The four folds of **RQ2** are too few for such a test. At the reported precision, two comparisons are equal (0.80 versus 0.80 and 0.958 versus 0.958). The unrounded differences, 0.004 for the **RQ1** Disjoint classifier over the held-out baseline and 0.0001 for Flakify+'s weighted F1 score over the always-non-flaky baseline, are far below the fold-to-fold variability of 0.02–0.08. The **RQ3** percentages are based on only 86 tests from 26 repositories and are correspondingly uncertain.

D. Construct Validity

We describe the degree to which our scales and metrics measure the intended properties [48].

On project-disjoint evaluation as a proxy for practical use. Our evaluation protocol holds out a set of projects as the test set on which the models are never trained. This simulates cross-project use but may not capture within-project temporal drift, which could be closer to practical use. However, we stress that our evaluation protocol aims to show that models struggle to learn more abstract features that underlie flakiness, instead relying on lexical features, as Camara et al. [25] already showed for the original lexical approach.

On the scope of code-based detectors. Our claim targets detection from the test source code. The classifiers we evaluated see only test-method tokens. One might argue that

production code called by the test, configuration files, or dependency manifests are also "code". Our construct draws the boundary at the test method, matching prior work but potentially excluding useful static signals. We argue, however, that our analysis of `FlakeCI` supports this boundary: While a potential dependency on an external service, DOM state, or setup step may be statically visible, deciding whether it explains a particular flaky failure requires execution-level evidence.

VII. Related Work

Benchmark and evaluation-validity critiques. Our work continues a line of recent studies questioning whether reported gains on code-based flakiness benchmarks reflect real detection ability. Analogous benchmark shortcuts are documented for other SE-ML tasks, such as deep-learning vulnerability detection [49]. Rahman et al. identified the cross-validation data-leakage flaw described in Section II [17]; correcting it and re-evaluating on a more realistic dataset (`FlakeBench`) with rerun-confirmed non-flaky labels lowered the measured performance. We found that the same flaw affected the leave-one-project-out evaluation. Berndt et al. argue more directly that the test code may not contain the information needed to distinguish flaky from non-flaky tests, finding that whether humans consider themselves able to label a test from its code depends on the flakiness type, with classes such as side effects of tested or utility code remaining invisible in the test itself [19]. Our contribution differs from both. Rahman et al. localized one implementation defect, and Berndt et al. showed through human labeling that the test code *may* lack the signal for certain flakiness classes. We instead show that the collapse to the baseline is not specific to one detector or one labeling: It recurs across the two published code-based detectors that permit project-disjoint evaluation and persists on `C-IDoFT`, a controlled counterfactual that removes the labeling shortcut while keeping the benchmark's flaky tests. We then move beyond showing that the code lacks the signal to locating it, finding that most flaky E2E failures cannot be diagnosed from the test source and the CI log alone and that, where they can, the decisive evidence sits in the CI run rather than in the test source.

Detection from signals other than test code. Much work detects flakiness without inspecting test code, which supports our thesis that the relevant signal is elsewhere. Several approaches model test execution history. Kowalczyk et al. ranked tests at Apple using entropy and flip-rate computed over pass/fail histories [7], and Herzig and Nagappan mined association rules over test-step results to flag false alarms [8]. Haben et al. argue that prediction should account for execution failures and target flaky *executions* rather than flaky tests [20]. Dynamic, rerun-based detectors such as `iDFlakies` identify flaky tests by reordering and re-executing test suites [29], and log-based techniques locate the root cause of flakiness by diffing the logs of passing and failing runs in industrial settings [50]. Surveyed developers nonetheless still asked for lightweight static detection support [51].

Empirical studies of flakiness causes. Root-cause studies attribute most flakiness to environmental rather than syntactic causes. Eck et al. had Mozilla developers classify previously fixed flaky tests and reported a taxonomy led by concurrency and async-wait issues, alongside newly identified categories such as overly restrictive value ranges [2]. Romano et al. found that UI-based (E2E) flaky tests are dominated by async waits and environment issues such as network, resource, and platform behavior [39]. These are causes a code-based detector largely cannot observe in the expensive E2E setting that **RQ3** targets. Such flakiness is costly in practice: An industrial CI case study attributed at least 2.5% of productive developer time to handling flaky failures [1].

Toward failure-level triage. Our forward-looking position, moving from labeling tests to classifying observed failures, aligns with work that operates at the execution level. Alshammari et al. classified whether an individual test *failure* is flaky from its failure output, such as stack traces, rather than labeling the test from its source [52], complementing the log-based root cause analysis of Lam et al. [50] and the alarm-level detection of Herzig and Nagappan [8]. Researchers at Google have trained a machine learning model to detect flaky failures and suppress re-execution of tests [3], and An et al. used failure symptoms, such as error messages and stack traces, to identify flaky failures in SAP HANA's CI pipeline [53]. Lampel et al. classified intermittent job failures from CI telemetry at Mozilla [54], and Aïdasso et al. prioritized flaky job-failure categories at TELUS [55]. Haben et al.'s study cited above, a collaboration with Ubisoft, Meta, and Broadcom, makes this case at industrial scale [20]. We view this execution-grounded framing, rather than code-based test-level detection, as the better-posed problem.

VIII. Conclusion

We conducted a study of code-based flaky test detection by reproducing three published LLM-based detectors (`Flakify`, `FlakyQ`, `FlakyXBert`) and evaluating them under progressively stricter protocols. Our reproduction revealed that the reported F1 scores of these detectors rest on evaluation shortcuts in the commonly used benchmarks, including data leakage across cross-validation folds, near-duplicate flaky/non-flaky test pairs introduced by the fix-commit labeling strategy, and project overlap between training and test splits. After we addressed these issues and evaluated in a project-disjoint setting, `Flakify` and `FlakyQ` collapsed to the level of a majority-class baseline, while rerunning `FlakyXBert`'s four affected notebooks on their own data lowered its weighted F1 score from 0.94 to 0.83, which stays above the always-flaky baseline of 0.66 in the within-project setting. We conclude that the high F1 scores reported in prior work reflect properties of the benchmarks rather than a learned ability to distinguish flaky from non-flaky tests.

To investigate this idea, we curated `C-IDoFT`, a controlled-counterfactual dataset of 54 468 tests from 57 projects that rebuilds the non-flaky class of the benchmark prior detectors were evaluated on, deriving non-flaky labels from 500 repeated

executions rather than fixed versions of flaky tests. Evaluating both CodeBERT variants on `C-IDoFT` in a project-disjoint setting confirmed the findings of our reproduction: Neither meaningfully outperformed the constant baselines. These results are consistent with prior replication studies of lexical code-based approaches without LLMs, which already showed limited generalizability on unseen projects [25], on newer revisions of the same projects [12], and on industrial data [13]. Our results suggest that replacing lexical features and traditional machine learning with LLMs does not overcome this limitation, as it is rooted in the task framing rather than in model capacity. Prompting GPT-4o-class LLMs classified flakiness from the test code only marginally better than random guessing [19].

We complemented this quantitative analysis with a root cause study of flaky E2E tests we curated from CI logs on GitHub Actions. Our manual classification of 86 flaky tests showed that for 58% of the tests, we could not attribute the failure to a cause from the test code or the CI log. Therefore, we conclude that the test code records what is asserted, not whether the runtime conditions for that assertion will hold on a given execution, which helps explain why code-based detectors struggle to reliably detect flakiness.

Our results relocate flaky-test prediction. While predicting flakiness from static code alone is appealing for its low cost, our results show that, for many kinds of flakiness, the test code does not contain the information this prediction requires. Instead of asking whether a *test* is flaky, we can ask whether an observed *failure* is flaky, or how likely a test is to fail given its execution environment, questions grounded in the execution-level evidence that our `FlakeCI` analysis found decisive in every diagnosed case. Our two datasets support work on these questions. `C-IDoFT` is a control on which static detectors no longer achieve inflated scores, and `FlakeCI` is a corpus of confirmed flaky E2E failures with the diagnostic evidence attached. We encourage the community to extend such corpora into failure-level benchmarks by adding genuine-failure counterexamples, which the same mining method can collect. We lay out concrete openings for this agenda in Section V, including failure-level classification and language-model agents that probe the execution environment directly. A nascent line of work already argues for predicting flaky *executions* rather than flaky *tests* [20, 52]. Our findings give that direction empirical support and new shared resources. How far are we from detecting flaky tests? Farther than reported F1 scores suggest. Under project-disjoint evaluation of developer-confirmed flakiness with rerun-confirmed non-flaky labels, code-based detectors are indistinguishable from a majority-class baseline, and for most flaky E2E tests we examined, the test code and CI log did not suffice to attribute the cause. The way forward, we argue, is to reframe the question at the level of executions.

References

[1] F. Leinen, D. Elsner, A. Pretschner, A. Stahlbauer, M. Sailer, and E. Jürgens, "Cost of flaky tests in continuous integration: An industrial case study," in *Proc. ICST*. IEEE, 2024, pp. 329–340.

[2] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: the developer's perspective," in *Proc. ESEC/FSE*. ACM, 2019, pp. 830–840.

[3] M. Hoang and A. Berding, "Presubmit rescue: Automatically ignoring flakytest executions," in *Proc. FTW*. ACM, 2024, pp. 1–2.

[4] N. Alshahwan, M. Harman, and A. Marginean, "Software testing research challenges: An industrial perspective," in *Proc. ICST*. IEEE, 2023, pp. 1–10.

[5] A. M. Memon, Z. Gao, B. N. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming google-scale continuous testing," in *Proc. ICSE-SEIP*. IEEE Computer Society, 2017, pp. 233–242.

[6] A. Berndt, T. Bach, and S. Baltes, "Do test and environmental complexity increase flakiness? an empirical study of SAP HANA," in *Proc. ESEM*. ACM, 2024, pp. 572–581.

[7] E. Kowalczyk, K. Nair, Z. Gao, L. Silberstein, T. Long, and A. M. Memon, "Modeling and ranking flaky tests at apple," in *Proc. ICSE-SEIP*. ACM, 2020, pp. 110–119.

[8] K. Herzig and N. Nagappan, "Empirically detecting false test alarms using association rules," in *Proc. ICSE*. IEEE Computer Society, 2015, pp. 39–48.

[9] V. Pontillo, F. Palomba, and F. Ferrucci, "Toward static test flakiness prediction: a feasibility study," in *Proc. MaLTeSQuE@ESEC/FSE*. ACM, 2021, pp. 19–24.

[10] J. Cai, Y. Lei, Z. Li, C. Liu, H. Xie, and Z. Wu, "Deepflaky: Deep hybrid representation learning for flaky test prediction," *Inf. Softw. Technol.*, vol. 193, p. 108070, 2026.

[11] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *Proc. MSR*. Seoul, Republic of Korea: ACM, 2020, pp. 492–502.

[12] G. Haben, S. Habchi, M. Papadakis, M. Cordy, and Y. L. Traon, "A replication study on the usability of code vocabulary in predicting flaky tests," in *Proc. MSR*. Madrid, Spain: IEEE, 2021, pp. 219–229.

[13] A. Berndt, Z. Nochta, and T. Bach, "The vocabulary of flaky tests in the context of SAP HANA," in *Proc. ESEM*. IEEE, 2023, pp. 1–9.

[14] J. Medeiros and B. Miranda, "A preliminary study on the vocabulary of flaky tests in swift," in *Proc. FTW*. ACM, 2026, pp. 11–12.

[15] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "Flakeflagger: Predicting flakiness without rerunning tests," in *Proc. ICSE*. Madrid, Spain: IEEE, 2021, pp. 1572–1584.

[16] S. Fatima, T. A. Ghaleb, and L. C. Briand, "Flakify: A black-box, language model-based predictor for flaky tests," *IEEE Trans. Software Eng.*, vol. 49, no. 4, pp. 1912–1927, 2023.

[17] S. Rahman, S. Dutta, and A. Shi, "Understanding and improving flaky test classification," *Proc. ACM Program.*

Lang., vol. 9, no. OOPSLA2, pp. 1345–1371, 2025.

[18] R. More and J. S. Bradbury, "An analysis of LLM fine-tuning and few-shot learning for flaky test detection and classification," in *Proc. ICST*. IEEE, 2025, pp. 349–359.

[19] A. Berndt, V. Bekmyradov, R. Gemulla, M. Kessel, T. Bach, and S. Baltes, "Can we classify flaky tests using only test code? an LLM-based empirical study," in *Proc. SANER, RENE Track*. Limassol, Cyprus: IEEE, 2026.

[20] G. Haben, S. Habchi, J. Micco, M. Harman, M. Papadakis, M. Cordy, and Y. L. Traon, "The importance of accounting for execution failures when predicting test flakiness," in *Proc. ASE*. ACM, 2024, pp. 1979–1989.

[21] S. Rahman, A. Baz, S. Misailovic, and A. Shi, "Quantizing large-language models for predicting flaky tests," in *Proc. ICST*. Toronto, ON, Canada: IEEE, 2024, pp. 93–104.

[22] Ö. O. Gültekin, A. Berndt, J. Bell, T. Bach, and S. Baltes, "How Far Are We from Detecting Flaky Tests? On the Limits of Code-Based Detection (Research Artifact)," Zenodo, Jul. 2026, https://doi.org/10.5281/zenodo.21102431.

[23] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 1, pp. 17:1–17:74, 2022.

[24] R. Verdecchia, E. Cruciani, B. Miranda, and A. Bertolino, "Know you neighbor: Fast static prediction of test flakiness," *IEEE Access*, vol. 9, pp. 76 119–76 134, 2021.

[25] B. H. P. Camara, M. A. G. Silva, A. T. Endo, and S. R. Vergilio, "What is the vocabulary of flaky tests? an extended replication," in *Proc. ICPC*. IEEE, 2021, pp. 444–454.

[26] S. Fatima, H. Hemmati, and L. C. Briand, "Flakyfix: Using large language models for predicting flaky test fix categories and test code repair," *IEEE Trans. Software Eng.*, vol. 50, no. 12, pp. 3146–3171, 2024.

[27] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "Deflaker: automatically detecting flaky tests," in *Proc. ICSE*. ACM, 2018, pp. 433–444.

[28] W. Lam, "International Dataset of Flaky Tests (IDoFT)," 2020, http://mir.cs.illinois.edu/flakytests.

[29] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A framework for detecting and partially classifying flaky tests," in *ICST 2019: 12th IEEE International Conference on Software Testing, Verification and Validation*, Xi'an, China, April 2019, pp. 312–322.

[30] A. Akli, G. Haben, S. Habchi, M. Papadakis, and Y. L. Traon, "FlakyCat: Predicting flaky tests categories using few-shot learning," in *Proc. AST*. IEEE, 2023, pp. 140–151.

[31] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proc. FSE*. ACM, 2014, pp. 643–653.

[32] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, ser. Findings of ACL, vol. EMNLP 2020. Association for Computational Linguistics, 2020, pp. 1536–1547.

[33] Association for Computing Machinery, "Artifact Review and Badging — Current," https://www.acm.org/publications/policies/artifact-review-and-badging-current, 2020, accessed: May 2026.

[34] M. Gruber, M. F. Roslan, O. Parry, F. Scharnböck, P. McMinn, and G. Fraser, "Do automatic test generation tools generate flaky tests?" in *Proc. ICSE*. ACM, 2024, pp. 47:1–47:12.

[35] A. Berndt, T. Bach, R. Gemulla, M. Kessel, and S. Baltes, "On the flakiness of llm-generated tests for industrial and open-source database management systems," *CoRR*, vol. abs/2601.08998, 2026.

[36] K. Ngo, V. Nguyen, and T. N. Nguyen, "Research on test flakiness: from unit to system testing," in *Proc. ASE*. ACM, 2022, pp. 218:1–218:4.

[37] A. Berndt, S. Baltes, and T. Bach, "Taming timeout flakiness: An empirical study of SAP HANA," in *Proc. ICSE-SEIP*. ACM, 2024, pp. 69–80.

[38] N. Hashemi, A. Tahir, and S. Rasheed, "An empirical study of flaky tests in javascript," in *Proc. ICSME*. IEEE, 2022, pp. 24–34.

[39] A. Romano, Z. Song, S. Grandhi, W. Yang, and W. Wang, "An empirical analysis of ui-based flaky tests," in *Proc. ICSE*. IEEE, 2021, pp. 1585–1597.

[40] T. Durieux, C. Le Goues, M. Hilton, and R. Abreu, "Empirical study of restarted and flaky builds on travis CI," in *Proc. MSR*. ACM, 2020, pp. 254–264.

[41] H. D. Dixit, S. Pendharkar, M. Beadon, C. Mason, T. Chakravarthy, B. Muthiah, and S. Sankar, "Silent data corruptions at scale," *CoRR*, vol. abs/2102.11245, 2021.

[42] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. ICML*. Pittsburgh, PA, USA: ACM, 2006, pp. 233–240.

[43] V. Terragni, P. Salza, and F. Ferrucci, "A container-based infrastructure for fuzzy-driven root causing of flaky tests," in *Proc. ICSE-NIER*. ACM, 2020, pp. 69–72.

[44] M. B. Brewer and W. D. Crano, "Research Design and Issues of Validity," *Handbook of research methods in social and personality psychology*, pp. 11–26, 2014.

[45] D. Silva, M. Gruber, S. Gokhale, E. Arteca, A. Turcotte, M. d'Amorim, W. Lam, S. Winter, and J. Bell, "The effects of computational resources on flaky tests," *IEEE Trans. Software Eng.*, vol. 50, no. 12, pp. 3104–3121, 2024.

[46] S. Baltes and P. Ralph, "Sampling in software engineering research: a critical review and guidelines," *Empir. Softw. Eng.*, vol. 27, no. 4, p. 94, 2022.

[47] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*, 2nd ed. Berlin, Heidelberg: Springer, 2024.

[48] P. Ralph and E. D. Tempero, "Construct validity in software engineering research and software metrics," in *Proc. EASE*. ACM, 2018, pp. 13–23.

[49] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" *IEEE Trans. Software Eng.*, vol. 48, no. 9, pp. 3280–3296, 2022.

[50] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *Proc. ISSTA*. ACM, 2019, pp. 101–111.

[51] M. Gruber and G. Fraser, "A survey on how test flakiness affects developers and what support they need to address it," in *Proc. ICST*. IEEE, 2022, pp. 82–92.

[52] A. Alshammari, P. Ammann, M. Hilton, and J. Bell, "230,439 test failures later: An empirical evaluation of flaky failure classifiers," in *Proc. ICST*. Toronto, ON, Canada: IEEE, 2024, pp. 257–268.

[53] G. An, J. Yoon, T. Bach, J. Hong, and S. Yoo, "Just-in-time flaky test detection via abstracted failure symptom matching," in *Proc. ICSME*. IEEE, 2024, pp. 741–752.

[54] J. Lampel, S. Just, S. Apel, and A. Zeller, "When life gives you oranges: detecting and diagnosing intermittent job failures at mozilla," in *Proc. ESEC/FSE*. ACM, 2021, pp. 1381–1392.

[55] H. Aïdasso, F. Bordeleau, and A. Tizghadam, "On the diagnosis of flaky job failures: Understanding and prioritizing failure categories," in *Proc. ICSE-SEIP*. IEEE, 2025, pp. 192–202.